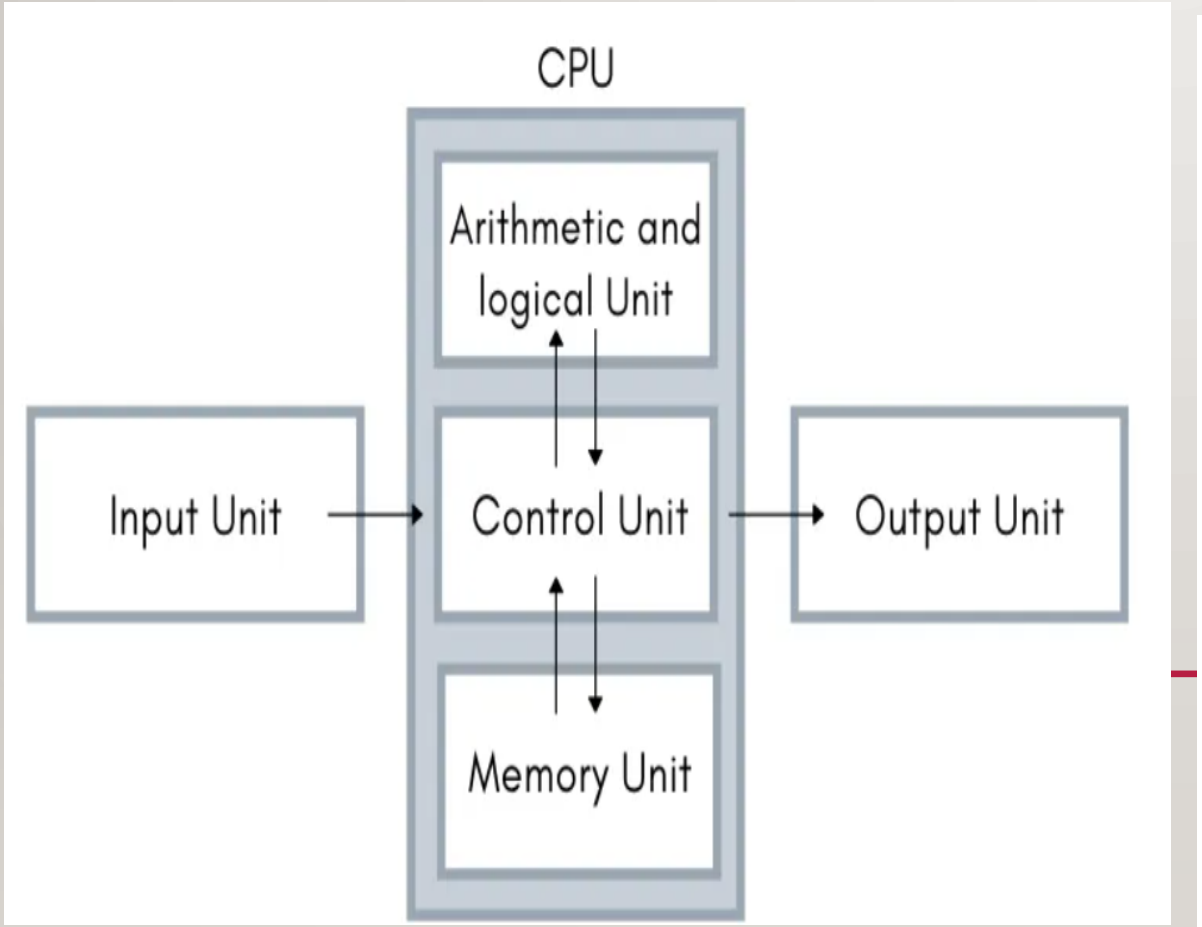
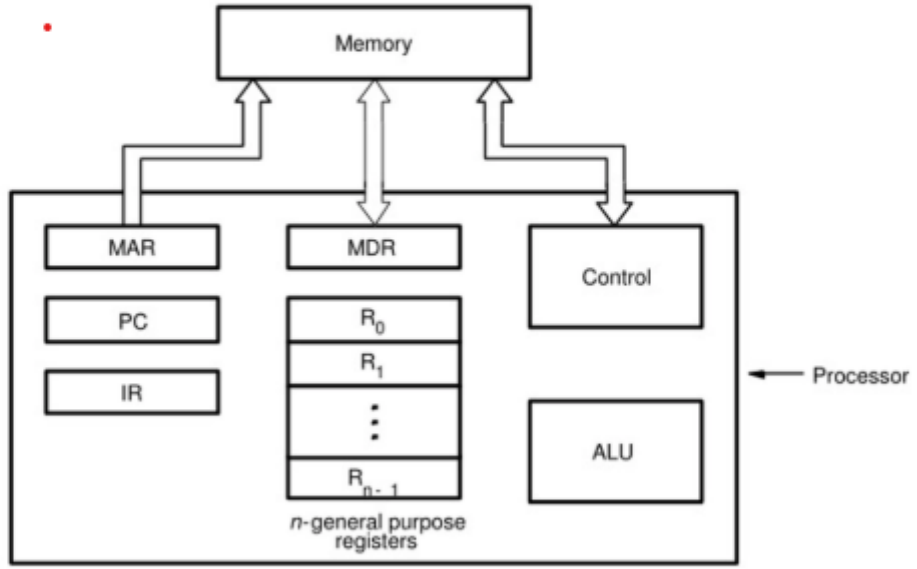
Fundamental Concepts

- Processor fetches one instruction at a time and perform the operation specified.
- Instructions are fetched from successive memory locations until a branch or a jump instruction is encountered.
- Processor keeps track of the address of the memory location containing the next instruction to be fetched using Program Counter (PC).
- Instruction Register (IR)



Connection Between the Processor and the Memory



Connections between the processor and the memory.

FUNDAMENTAL CONCEPTS

UNIT 2

Register

The registers are an integral part of the CPU.

They are a type of **memory** that can be accessed very quickly compared to other types of memory. The pieces of information they hold are needed very often. They can be used to **store data** and **control information** during a fetch-decode-execute cycle or they can be used to hold values that are generated as part of the ALU working on data.

SOME OF THE REGISTERS

- **Program Counter (PC)**
- **Instruction Register (IR)**
- **Memory Address Register (MAR)**
- **Memory Data Register (MDR)**
- **General Purpose Registers**

❖ General Purpose Registers

A register is a collection of flip-flops. Single bit digital data is stored using flip-flops. By combining many flip-flops, the storage capacity can be extended to accommodate a huge number of bits. We must utilize an n-bit register with n flip flops if we wish to store an n-bit word.

General Purpose Registers (GPRs) are essential components within a CPU, serving as temporary storage locations for data that is actively being processed. Unlike special-purpose registers, which are dedicated to specific tasks, GPRs are versatile and can store a wide variety of data, including operands for arithmetic and logic operations, memory addresses, or intermediate results.

They provide fast access to frequently used data, reducing the need to fetch from slower main memory and thereby speeding up program execution. Their contents can be read/written by programmer /user through instructions.

❖PROGRAM COUNTER

The program counter, also known as the **instruction pointer** or simply PC, is a fundamental component of a computer's central processing unit (CPU). It is a special register that keeps track of the memory address of the next instruction to be executed in a program.

❖INSTRUCTION REGISTER

The **Instruction Register (IR)** is a crucial component of a computer's Central Processing Unit (CPU). It holds the instruction currently being executed or decoded. The IR is part of the control unit of the CPU and plays a vital role in the instruction cycle, which includes fetching, decoding, and executing instructions.

Function and Workflow

The instruction register temporarily stores the instruction fetched from memory. This instruction is then decoded to determine the operation to be performed and the operands involved. The decoded instruction is executed by the CPU, and the results are stored back in memory or other registers.

Step-by-Step Process

1. **Fetch Phase**: The CPU fetches the next instruction from memory and loads it into the instruction register. This step ensures that the CPU knows what operation to execute next.
2. **Decode Phase**: The instruction in the IR is decoded to interpret the command. This involves determining the operation and the operands.
3. **Execute Phase**: The decoded instruction is executed. For example, an addition operation would add the contents of two registers and store the result in a third register.

❖Memory Address Register (MAR)

The **Memory Address Register (MAR)** is a crucial component within a computer's CPU. It is responsible for storing the memory address from which data will be fetched to the CPU registers or the address to which data will be sent and stored via the system bus. Essentially, the MAR holds the memory location of data that needs to be accessed during the execution phase of an instruction.

KEY FUNCTIONS OF MAR

- **Fetching Data**: When the CPU needs to read data from memory, the MAR holds the address of the data to be fetched.
- **Storing Data**: When the CPU needs to write data to memory, the MAR holds the address where the data will be stored.
- **Coordination with MDR**: The MAR works in conjunction with the Memory Data Register (MDR) to facilitate the transfer of data between the CPU and memory.

❖Memory Data Register (MDR)

The Memory Data Register (MDR) The Memory Data Register (MDR) is a **crucial component in CPU architecture**, serving as a buffer for data during transfer to and from main memory. It works with the Memory Address Register (MAR) to facilitate read and write operations, influencing the speed and efficiency of data processing.

Executing an Instruction

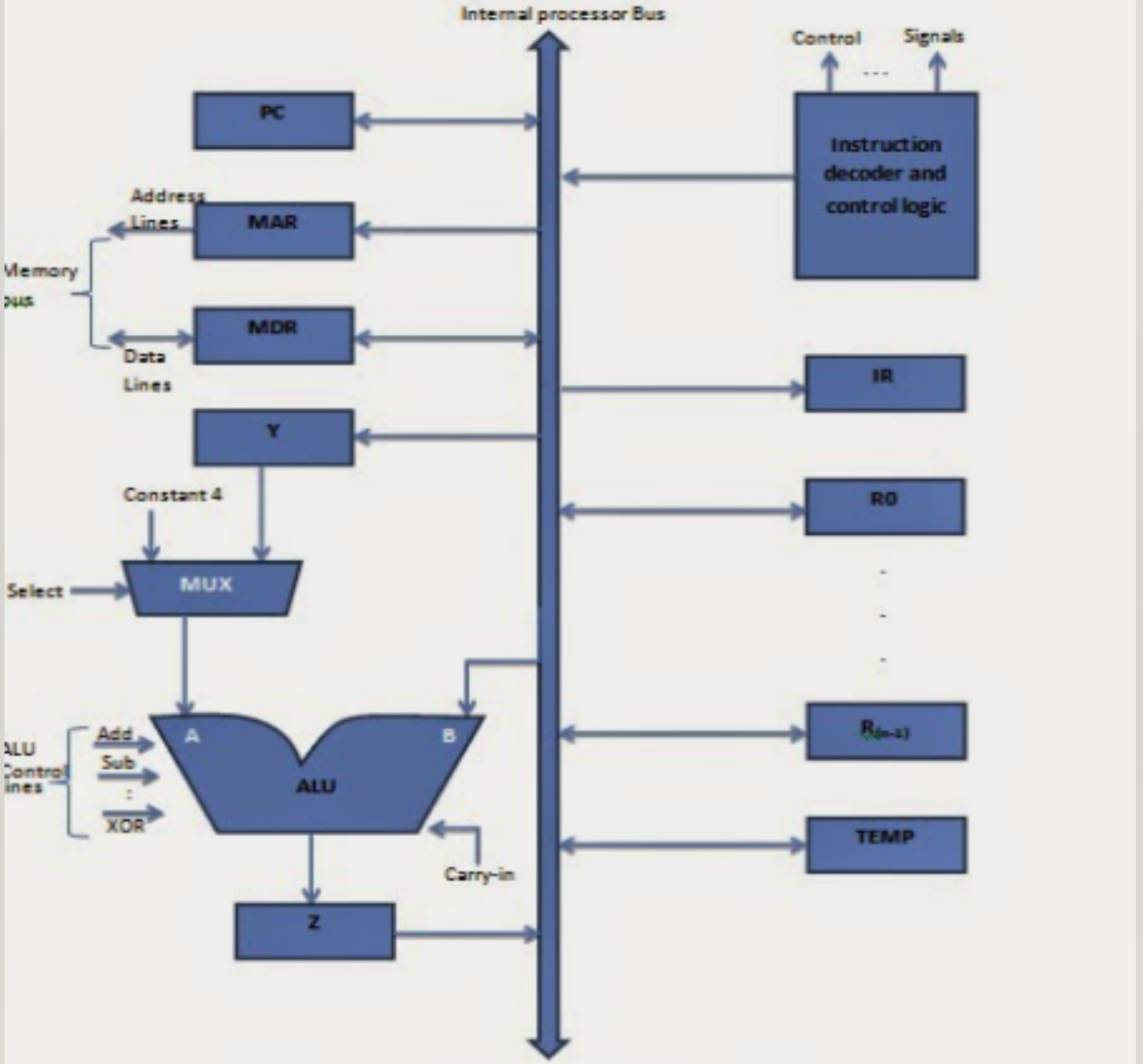
- Fetch the contents of the memory location pointed to by the PC. The contents of this location are loaded into the IR (fetch phase).

$$IR \leftarrow [[PC]]$$

- Assuming that the memory is byte addressable, increment the contents of the PC by 4 **(Fetch phase)**.

$$PC \leftarrow [PC] + 4$$

- Carry out the actions specified by the instruction in the IR **(Execution phase)**.



SINGLE BUS ORGANISATION

Single Bus Organization

- ALU
- Registers for temporary storage
- Various digital circuits for executing different micro operations.(gates, MUX, decoders, counters).
- Internal path for movement of data between ALU and registers.
- Driver circuits for transmitting signals to external units.
- Receiver circuits for incoming signals from external units.

Single Bus Organization - contd.

Program Counter (PC)

- Keeps track of execution of a program
- Contains the memory address of the next instruction to be fetched and executed.

Memory Address Register (MAR)

- Holds the address of the location to be accessed.
- I/P of MAR is connected to Internal bus and an O/P to external bus.

Memory Data Register (MDR)

- It contains data to be written into or read out of the addressed location.
- It has 2 inputs and 2 outputs.
- Data can be loaded into MDR either from memory bus or from internal processor bus.
- The data and address lines are connected to the internal bus via MDR and MAR

Single Bus Organization - contd.

Registers

- The proœssor registers R_0 to R_{n-1} vary considerably from one processor to another.
- Registers are provided for general purpose used by programmer.
- Special purpose registers-index & stack registers.
- Registers Y, Z & TEMP are temporary registers used by processor during the execution of some instruction.

Multiplexer

- Select either the output of the register Y or a constant value 4 to be provided as input A of the ALU.
- Constant 4 is used by the processor to increment the contents of PC.

Single Bus Organization - contd.

ALU

- B input of ALU is obtained directly from processor-bus.
- As instruction execution progresses, data are transferred from one register to another, often passing through ALU to perform arithmetic or logic operation.

Data Path

- The registers, ALU and interconnecting bus are collectively referred to as the data path.

❖DATA PATH

A **data path** (or datapath) is a critical component of a computer's architecture, consisting of various functional units such as:

- **Arithmetic Logic Units (ALUs)**: Perform arithmetic and logical operation s.
- **Registers**: Temporary storage locations for data being processed.
- **Buses**:Communication pathways that transfer data between components wi thin the CPU.

An Instruction Can Be Executed By Performing One Or More Of The Following Operations

- **Transfer a word of data from one processor register to another or to the ALU**
- **Perform an arithmetic or a logic operation and store the result in a processor register**
- **Fetch the contents of given memory location and load them into processor register**
- **Store a word of data from a processor register into a given memory location**

- **An Instruction code is a group of bits that instructs the computer to perform a specific operation.**

- **It is divided into two parts and the most important parts of an instruction code is its operation code or opcode. The second part is the address part which specifies the memory address.**

1. Register Transfers

- The input and output gates for register Ri are controlled by signals is Ri_{in} and Ri_{out}
  - Ri_{in} is set to 1 – data available on common bus are loaded into Ri.
  - Ri_{out} is set to 1 – the contents of register are placed on the bus.
  - Ri_{out} is set to 0 – the bus can be used for transferring data from other registers .
- All operations and data transfers within the processor take place within time-periods defined by the processor clock.
- When edge-triggered flip-flops are not used, 2 or more clock-signals may be needed to guarantee proper transfer of data. This is known as multiphase clocking.

Data Transfer Between Two Registers

Example:

- Transfer the contents of R1 to R4.
- Enable output of register R1 by setting $R1_{out}$=1. This places the contents of R1 on the processor bus.
- Enable input of register R4 by setting $R4_{in}$=1. This loads the data from the processor bus into register R4.

Input and Output Gating for Registers

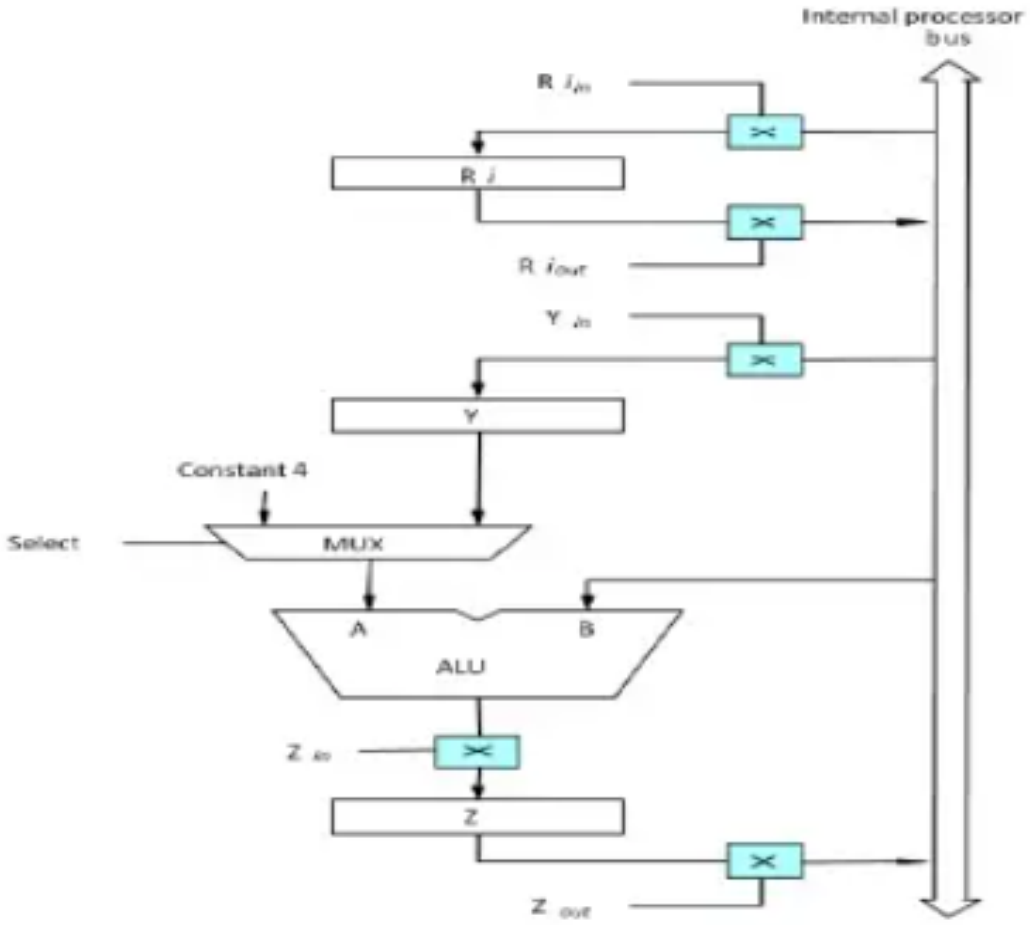


Figure. Input and Output Gating for the Registers

Input and Output Gating for One Register Bit

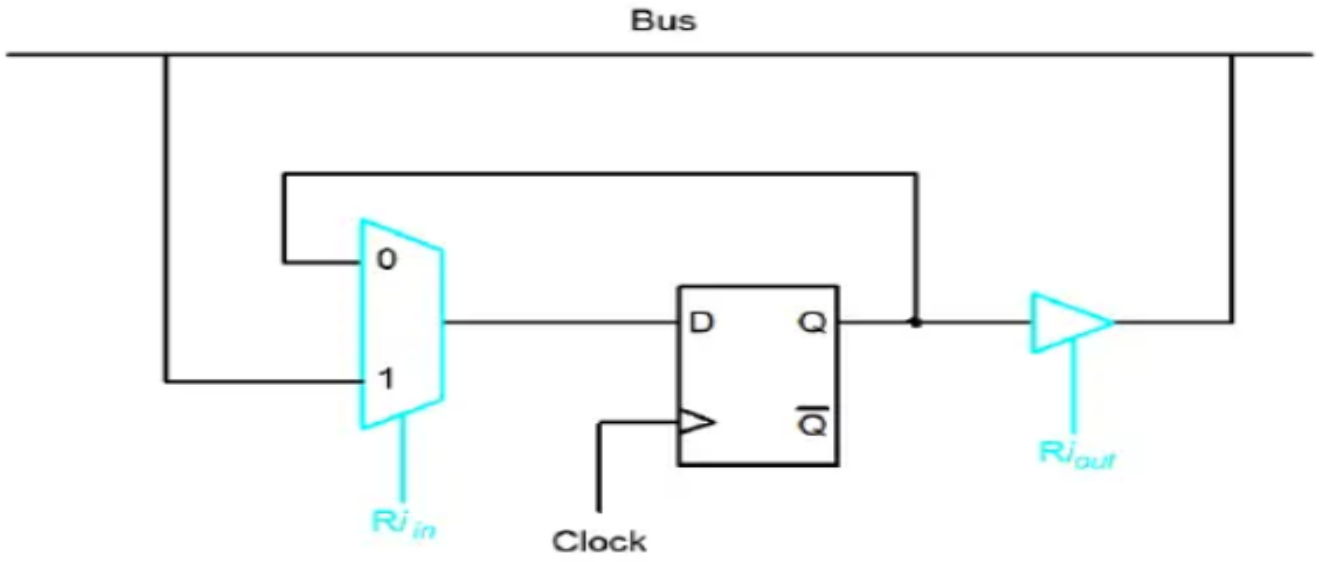
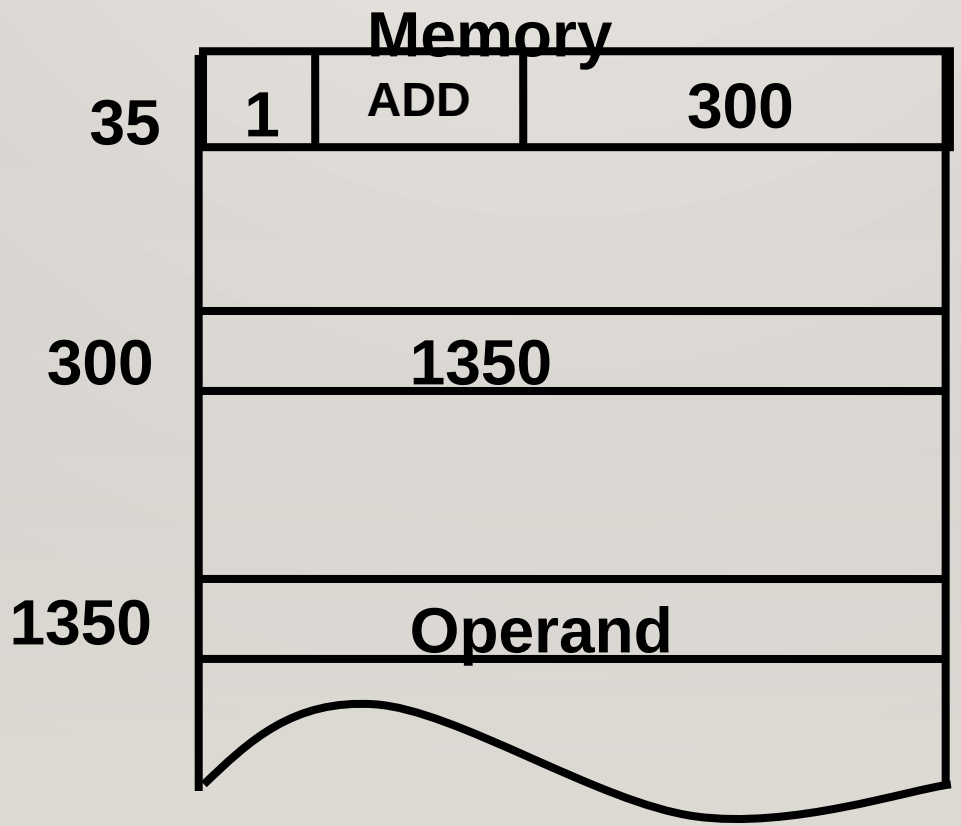


Figure. Input and Output Gating for One Register Bit

Input and Output Gating for One Register Bit - contd

- A 2-input multiplexer is used to select the data applied to the input of an edge-triggered D flip-flop.
  - When Ri_{in}=1, mux selects data on bus. This data will be loaded into flip-flop at rising-edge of clock.
  - When Ri_{in}= 0, mux feeds back the value currently stored in flip-flop.
- Q output of flip-flop is connected to bus via a tri-state gate.
  - When Ri_{out}= 0, gate's output is in the high-impedance state. (This corresponds to the open circuit state of a switch).
  - When Ri_{out}=1, the gate drives the bus to 0 or 1, depending on the value of Q.

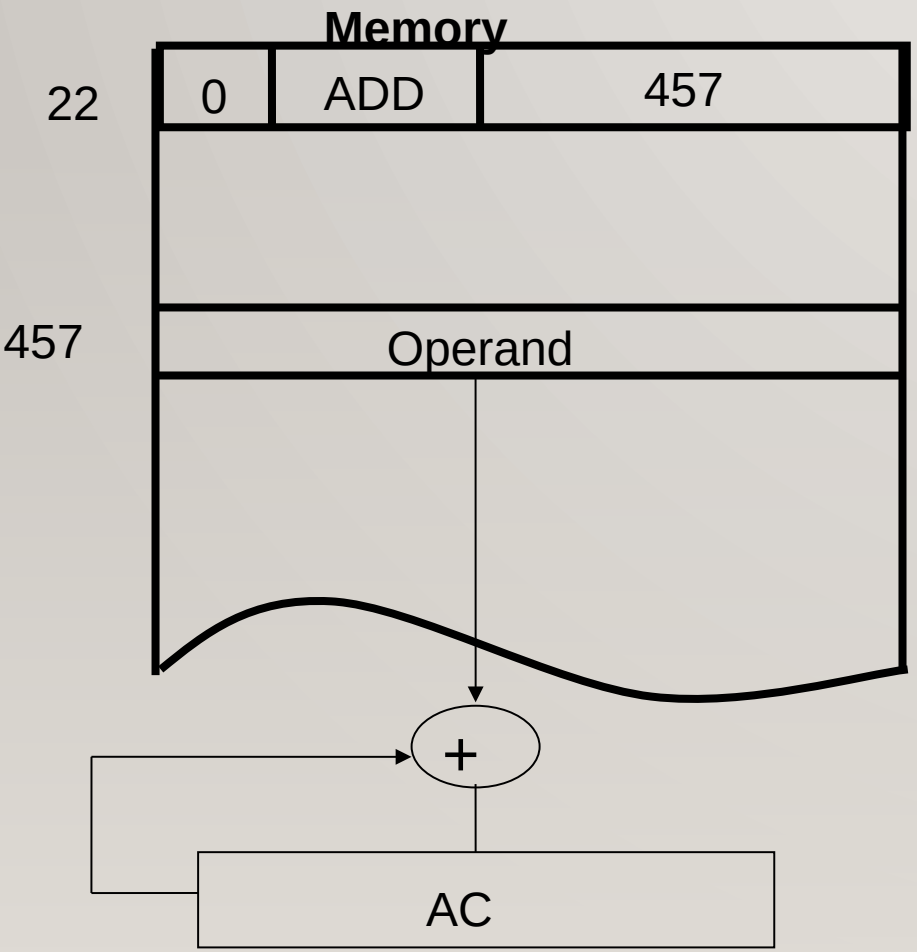

Memory
35
1
ADD
300
300
1350
1350
Operand

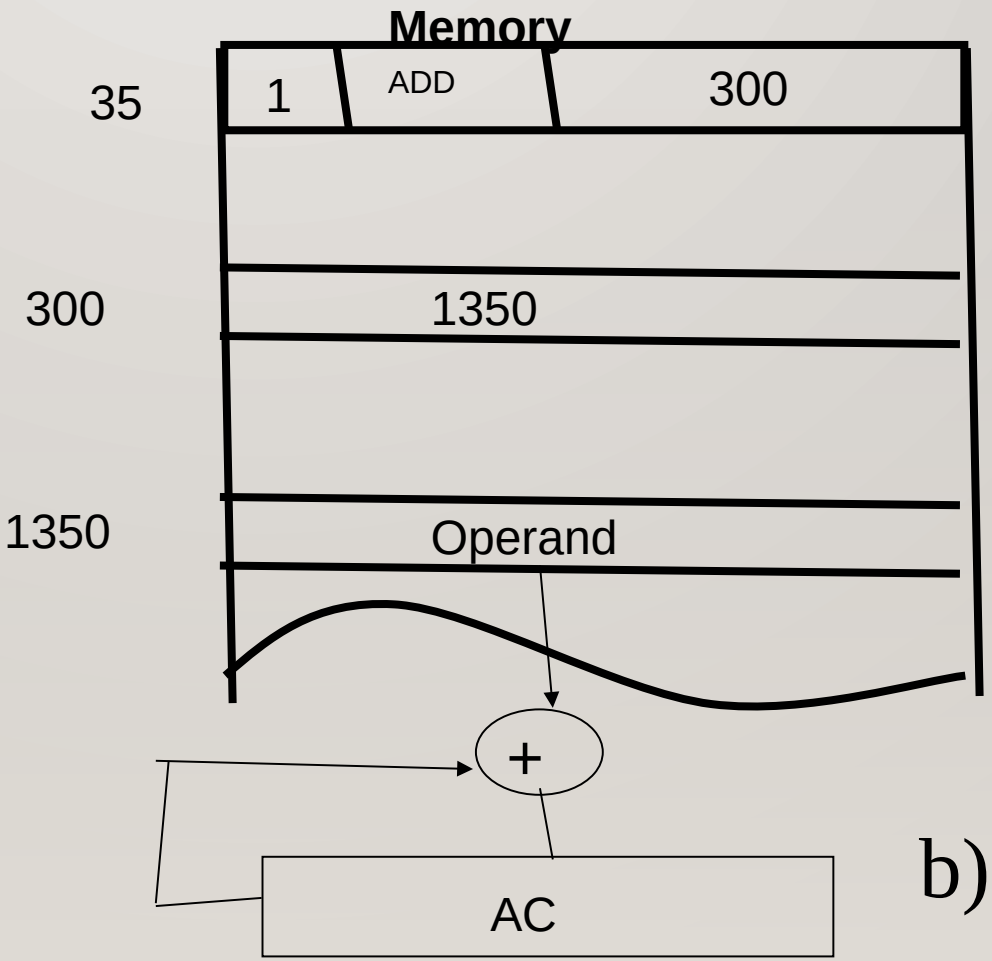
- **One bit of instruction code can be used to distinguish between a direct and indirect address. This configuration is shown below:**

- **An indirect address instruction is shown below:**

- **The indirect address instruction needs two references to memory to fetch an operand.**
  - **The first reference is needed to read the address of the operand; (i e to fetch the effective address)**
  - **And second is for the operand itself. (i.e. to read or write the actual operand.)**

EA=ADDRESS OF THE OPERAND





b) Indirect address

- The instruction in address 35 is shown in fig. above has a mode bit $I = 1$.
- Therefore, it is recognized as an indirect address instruction.
- The address part is the binary equivalent of 300.

- The control goes to address 300 to find the address of the operand.
- The address of the operand in this case is 1350.
- If the operand is found in address 1350, then it is added to the content of *AC*.

COMPUTER INSTRUCTIONS

- The basic computer has three instruction code formats as shown below:

1. Memory-Reference Instructions
2. Register-Reference Instructions
3. Input-Output Instructions

1. MEMORY-REFERENCE INSTRUCTION

- **A memory-reference instruction uses 12 bits to specify an address and one bit to specify the addressing mode *I*.**

- ***I*** **is equal to 0 for direct address and 1 for indirect address**

(Opcode is=000 through 110)

Example1:- AND 0122

AND 0 122

0 000 0001 0010 0010

I(Direct Addr) opcode Memory addrs

Example2:- AND 0122

AND 1 122

1 000 0001 0010 0010

I(Direct Addr) opcode Memory addrs

Hexadecimal Code

SYMBOL	*I* = 0	*I* = 1	DESCRIPTION
AND	0xxx	8xxx	AND memory word to *AC*
ADD	1xxx	9xxx	Add memory word to *AC*
LDA	2xxx	Axxx	Load memory word to *AC*
STA	3xxx	Bxxx	Store content of *AC in memory*
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and Save return address
ISZ	6xxx	Exxx	Increment and skip if zero.

- The hexadecimal code is equal to the equivalent hexadecimal number of the binary code used for the instruction.

- By using the hexadecimal equivalent we reduce the 16 bit of an instruction code to four digit with each hexadecimal digits being equal to four bits.

- When *I* = 0, the last four bits of an instruction have a hexadecimal digit equivalent from 0 to 6 since the last bit is 0.

- [0xxx – 6xxx i.e. 0000 x x x – 0110 x x x. So the last bit is zero.] So it's a *direct memory-reference instruction*.

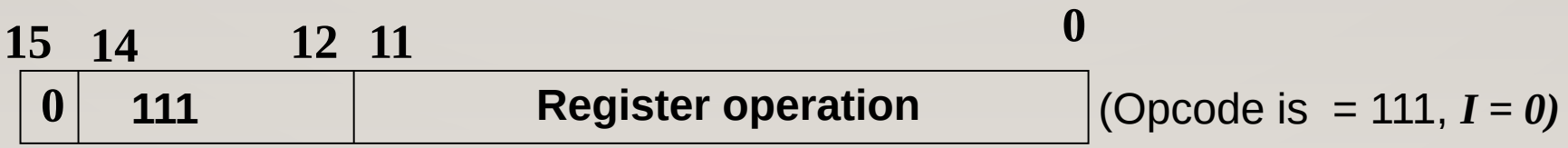
- **When *I = 1*, the hexadecimal digit equivalent of the last four bits of the instruction ranges from 8xxx to Exxx since the last bit is 1.**

- **[8xxx – Exxx i.e. 1000 x x x – 1110 x x x. So the last bit is one.] So it's an *indirect memory-reference instruction.***

2. REGISTER-REFERENCE INSTRUCTION

- **A register-reference instruction are recognized by the opcode 111 with a 0 in the leftmost bit (bit 15) of the instruction.**

- **A register-reference instruction specifies an operation on or a test on the *AC* register.**



(Opcode is = 111, ***I = 0)***

- An operand from memory is not needed and therefore the other 12 bits are used to specify the operation or test to be executed on registers.

7800
0111 1000 0000 0000

- Example:-

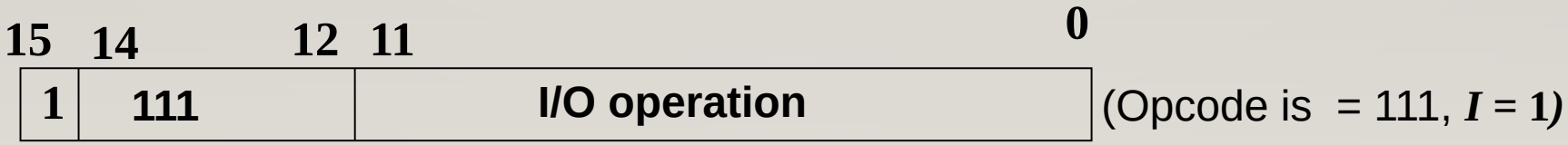
SYMBOL	Hexadecimal Code	DESCRIPTION
CLA	7800	Clear *AC*
CLE	7400	Clear *E*
CMA	7200	Complement *AC*
CIR	7080	Circulate right AC and E
INC	7020	Increment AC
HLT	7001	Halt computer

- Register reference instructions use 16 bits to specify an operation.
- The left most bit is always 0111, which is equal hexadecimal 7.
- The other three hexadecimal digits give the binary equivalent of the remaining 12 bits.

3. INPUT - OUTPUT INSTRUCTION

- **An input-output instruction does not need a reference to memory and is recognized by the operation code 111 with a 1 in the left most bit of the instruction.**
-

- **The remaining 12 bits are used to specify the type of input-output operation or test to be performed.**



(Opcode is = 111, ***I*** = **1*)***

- The input-output instructions also use all 16 bits to specify an operation .
- The last four bits are always 1111, equivalent to hexadecimal F.

- **The type of instruction is recognized by the computer from the four bits in positions 12 through 15 of the instruction.**

- If the three opcode bits in positions 12 through 14 are not equal to 111, the instruction is a memory-reference type and the bit in position 15 is taken as the addressing mode *I*.

- If the three bit opcode is equal to 111, control then inspects the bit in position 15.
- If this bit is 0, then the instruction is a regiter-reference type.
- If the bit is 1, the instruction is an input-output type.

SYMBOL	Hexadecimal Code	DESCRIPTION
INP	F800	Input character to AC
OUT	F400	Output character from AC
SKI	F200	Skip on input Flag
SKO	F100	Skip on output Flag
ION	F080	Interrupt ON
IOF	F040	Interrupt OFF

INSTRUCTION CYCLE

- **A program residing in memory unit of the computer consist of a sequence of instructions.**

- **The program is executed by going through a cycle for each instruction.**

- **Each instruction cycle consist of the following phases:**

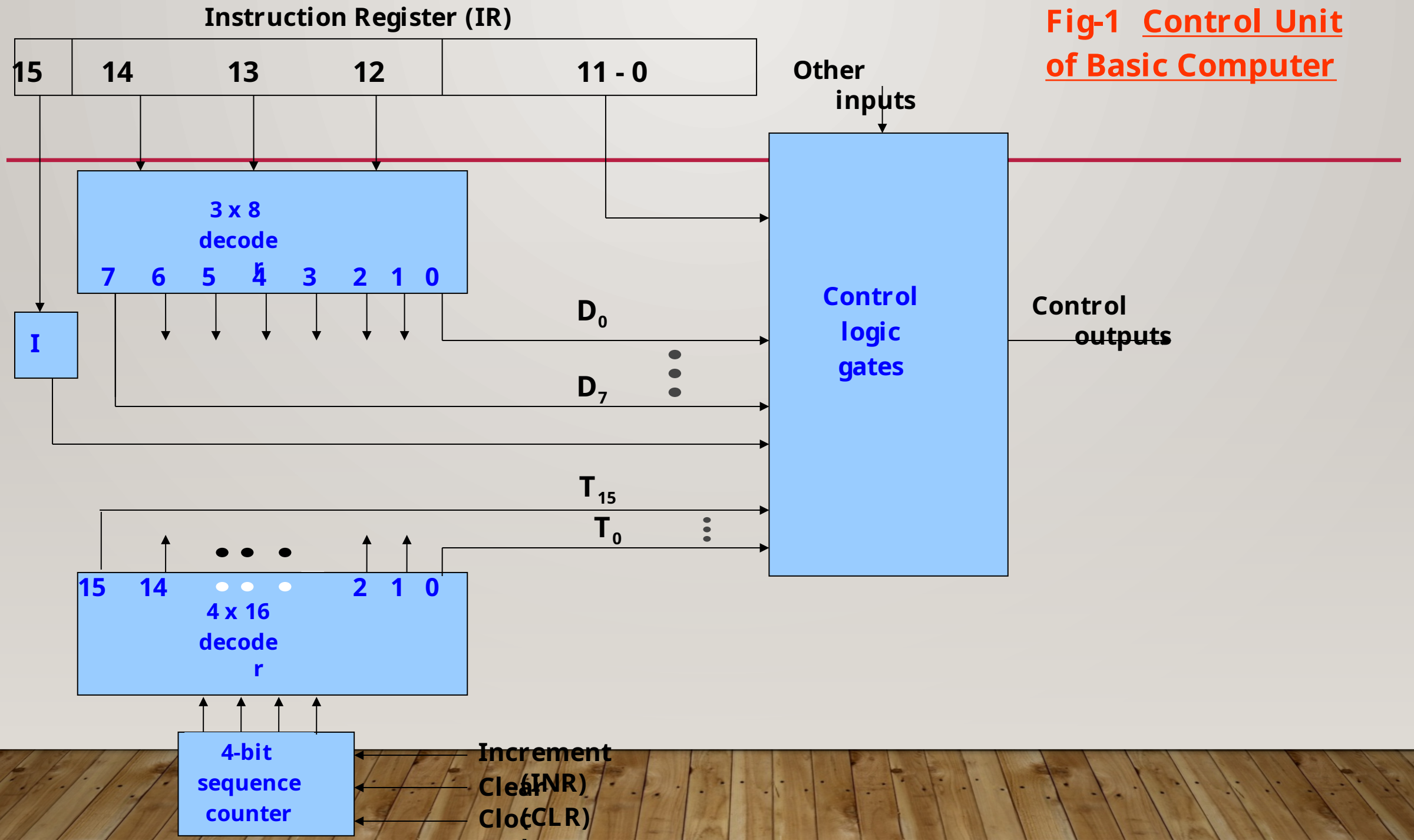
1. **Fetch an instruction from memory**
2. **Decode the instruction**
3. **Read the effective address from memory if the instruction has an indirect address.**
4. **Execute the instruction.**

CONTROL UNIT OF A BASIC COMPUTER

- **The block diagram of a control unit is shown below.**

- **It consist of two decoders (3x8 and 4x16), a sequence counter (SC), and a number of control logic gates.**

Fig-1 Control Unit of Basic Computer



- Upon the completion of step 4, the control goes back to step 1 to fetch, decode, and execute the next instruction.

- This process continues indefinitely unless a HALT instruction is encountered.

FETCH AND DECODE

- **Initially, the program counter is loaded with the address of the first instruction in the program.**

- **The sequence counter SC is cleared to 0, providing a decoding timing signal T_0.**

- After each clock pulse, SC is incremented by one, so that the timing signals go through a sequence T_0, T_1, T_2 and so on.

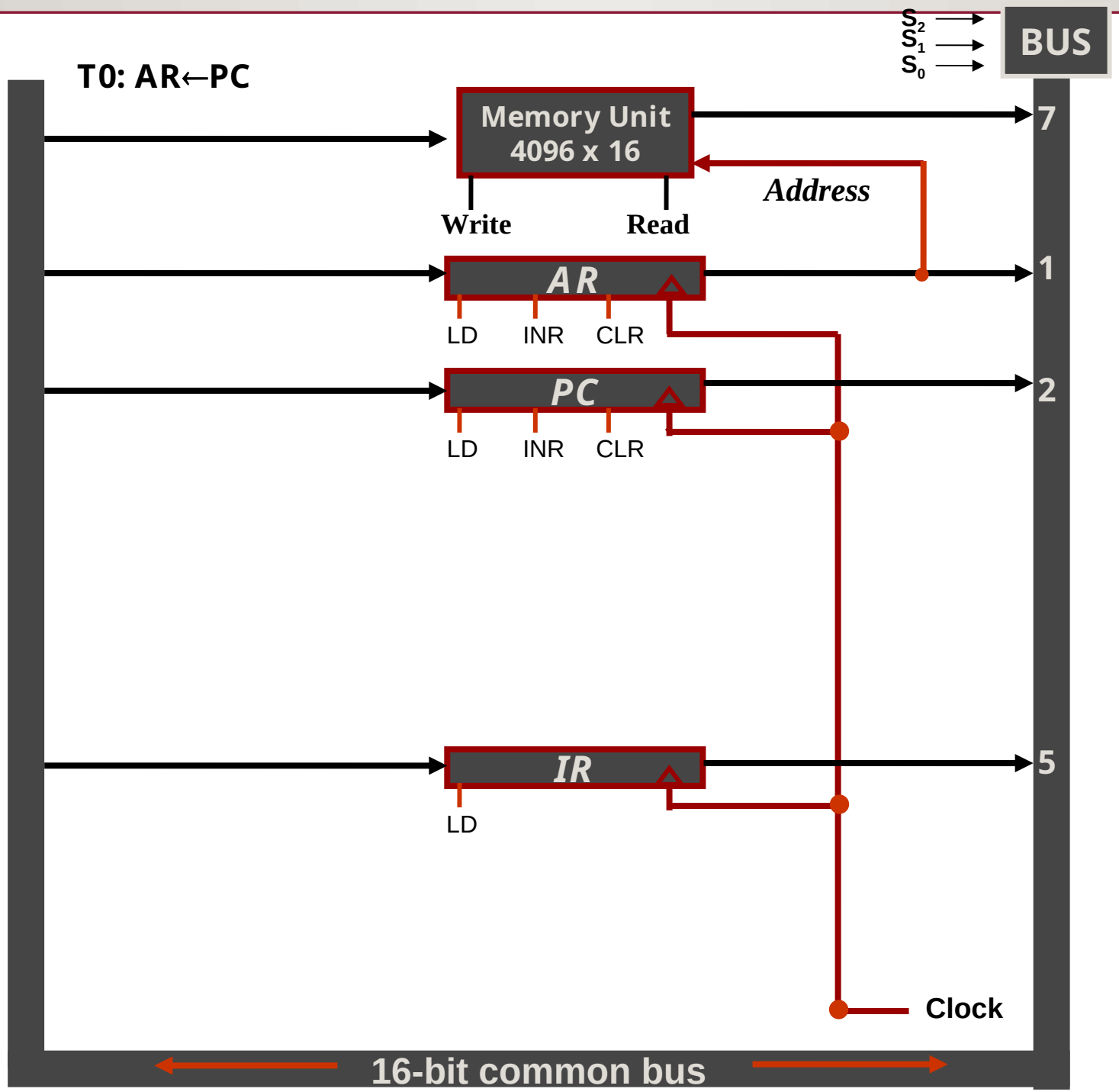
- The microoperations for the fetch and decode phases can be specified by the following register transfer statements.

- T_0: AR ← PC
- T_1: IR ← M[AR], PC ←PC+1
- T_2: D0, … D7←Decode IR(12-14), AR←IR(0-11), I ← IR(15)

❖ **To provide the data path for the transfer of PC to AR the following signals are required.**

1. **Place the content of PC onto the bus by making the bus selection inputs $S_2S_1S_0 = 010$.**

2. **Transfer the content of the bus to AR by enabling the LD input of AR.**


T0: AR←PC
S2
S1
S0
BUS
Memory Unit
4096 x 16
Write
Read
Address
7
AR
LD
INR
CLR
1
PC
LD
INR
CLR
2
IR
LD
5
Clock
16-bit common bus

- **In order to implement the second statement**

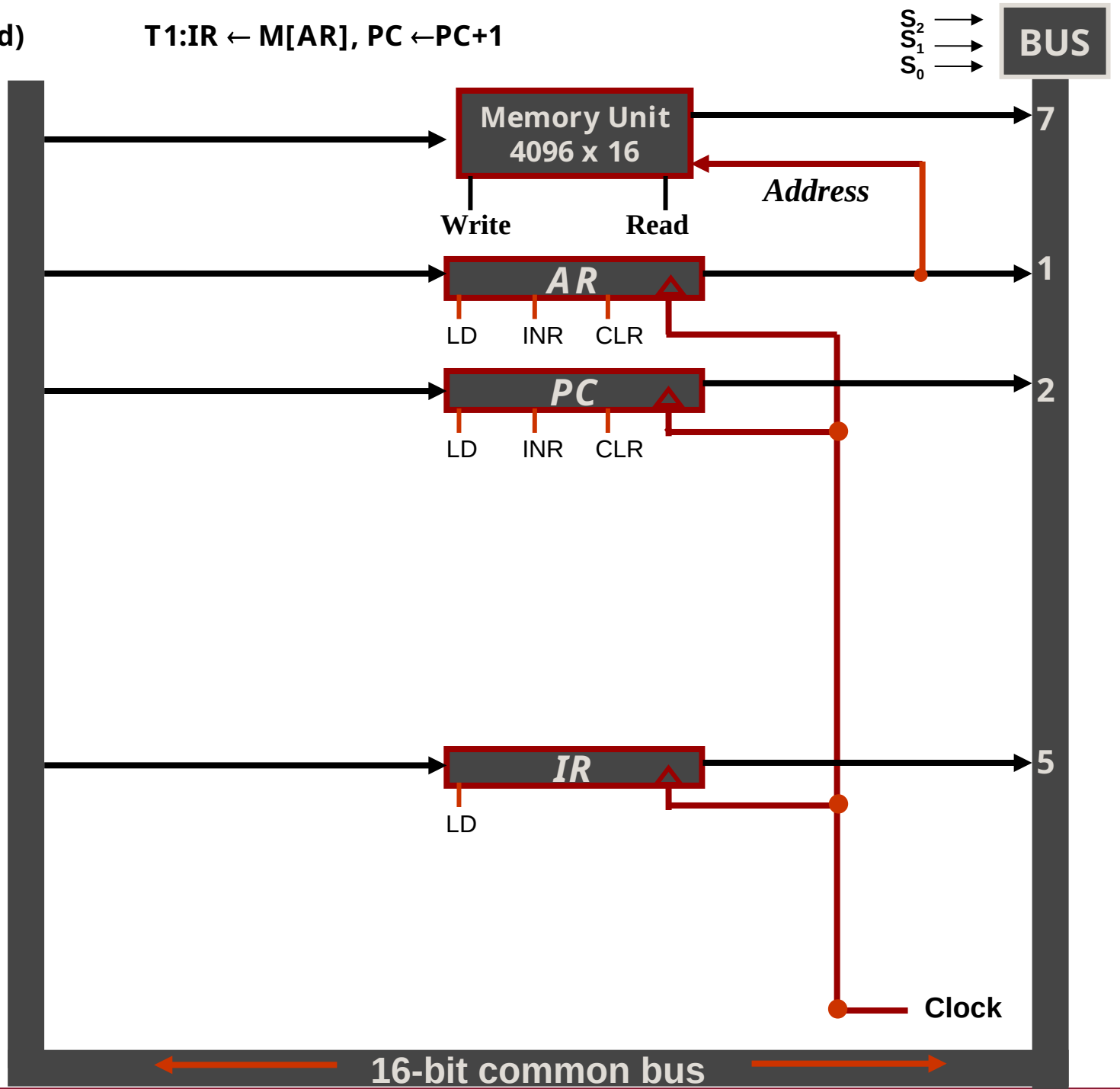
(Read) T_1**:IR ← M[AR], PC ←PC+1**

- **It is necessary to use the timing signal T1 to provide the following connections in the bus system.**

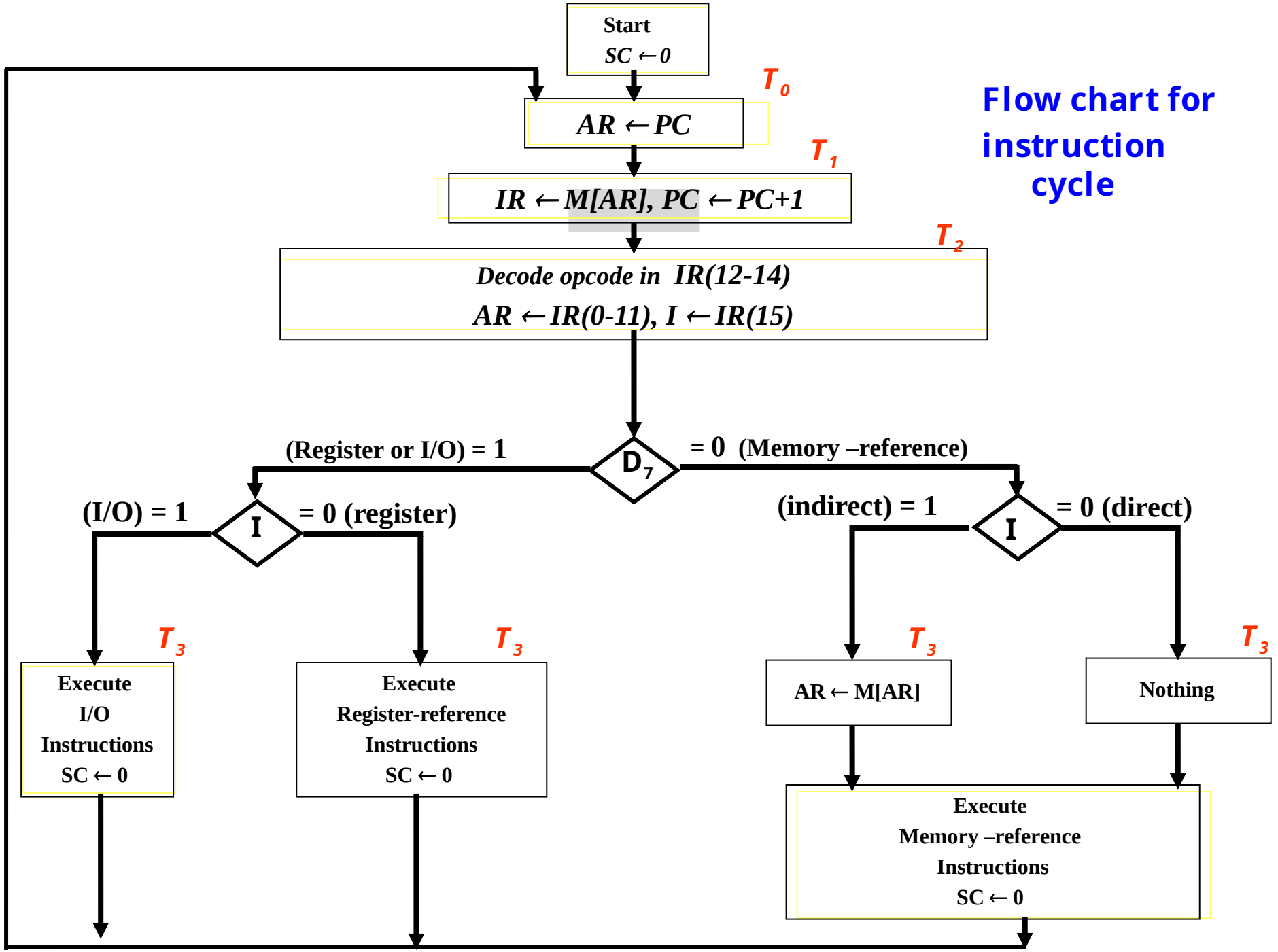
1. Enable the read input of memory
2. Place the content of memory onto the bus by making $S_2S_1S_0 = 111$.
3. Transfer the content of the bus to IR by enabling the LD input of IR.
4. Increment PC by enabling the INR input of PC.

T_1: IR ← M[AR], PC ← PC+1

(Read) T1:IR ← M[AR], PC ←PC+1



Flow chart for instruction cycle



Determine the type of Instruction

- The timing signal that is active after the decoding is T_3.
- During T3, the CU determines the type of instruction that was read from the memory.
- The flow chart above shows how the control determines the instruction type after decoding.

- The decoder output D_7 is equal to 1, if the opcode is equal to the binary 111.
- If $D_7 = 1$, the instruction must be register-reference or I/O type.
- If $D_7 = 0$, the instruction is recognized as a memory –reference instruction.

- Control then inspects the value of the first bit of instruction, which is available in flip-flop *I*.
- If $D_7 = 0$ and $I = 1$, its an indirect memory reference.
- It is then necessary to read the effective address from the memory.

- The microoperation for the indirect address condition in register transfer statement is as follows:

$$AR \leftarrow M[AR]$$

- Initially AR holds the address part of the instruction.
- This address is used during the memory read operation.

- The word at address given by AR i.e. M[AR] is read from memory and placed on the common bus

- The LD input of AR is then enabled to receive the indirect address that resided in the 12 least significant bits of the memory word.

END

REGISTER TRANSFERS

- In computer architecture, register transfer refers to the process of copying data between registers, which are small, fast storage locations within the CPU.
- It involves the use of a register transfer language (RTL), a symbolic notation for describing these data movements and the micro-operations (like addition or shift) performed on the data.

- A register transfer statement, such as R1 -> R2, indicates that the contents of register R1 are copied into R2. This action requires control signals to execute and data lines to facilitate the simultaneous, parallel transfer of bits during a single clock pulse, without altering the source register.

KEY CONCEPTS

- **Registers**:
- Small, high-speed storage units within a processor used to hold data for immediate processing.
- **Micro-operations**:
- The basic operations performed on the data stored in registers, such as data transfer, addition, subtraction, or shifting.
- **Register Transfer Language (RTL)**:
- A symbolic language used to describe the micro-operations and the flow of data between registers in a digital system.
- **Control Functions**:
- Control signals that initiate or allow a register transfer to occur only when a specific condition is met (e.g., P: R1 -> R2 only happens when control signal P is 1).

HOW REGISTER TRANSFER WORKS?

- **Source and Destination**:

  A register transfer involves a source register (where data is read from) and a destination register (where data is written to).

- **Parallel Transfer**:

  All bits of data from the source register are copied to the destination register simultaneously during one clock pulse.

- **Non-Destructive**:

  The transfer is non-destructive, meaning the content of the source register remains unchanged after the data is copied.

- **Hardware Implementation**:

  Register transfers require specific hardware logic circuits, including data lines connecting the registers and control lines to manage the transfer process.

- **Control**:

  A control function or signal determines when the transfer takes place, allowing for conditional execution of operations.

EXAMPLE

- R2 ← R1: This RTL statement signifies that the data stored in register R1 is copied into register R2.
- P: R1 → R2: This means that the contents of R1 are transferred to R2 only when the control signal P is active (P = 1).